

which returns a `String`, which becomes the text.

Event handling in a `JComboBox` is tricky for several reasons. When their selected item changes—either by being selected, or by having them be edited— every `JComboBox` fires two events.

1. For deselecting the previous item.
2. For selecting the current item.

Also, whenever an item event is fired, an action event is also fired right away.

Because Combo Boxes can be edited, they can fire `ActionEvents` that you can track with an `ActionListener`. In fact, whenever the currently displayed value is edited, the combo box fires an action event.

```
public class Test extends JApplet
{
    private JComboBox comboBox = new JComboBox();
    private ComboBoxEditor editor = comboBox.getEditor();
    public void init()
    {
        Container c = getContentPane();
        comboBox.setEditable(true);
        comboBox.addItem("Top");
        comboBox.addItem("Center");
        comboBox.addItem("Bottom");
        c.setLayout(new FlowLayout());
        c.add(comboBox);
        editor.addActionListener( new ActionListener()
        {
            public void actionPerformed(ActionEvent e)
            {
                String s = (String) editor.getItem();
                showStatus("Item Edited: " + s);
            }
        });
    }
}
```

This is an interface, it has method `getItem()`, which returns an `Object`. Once we get an `Object` out of the editor, we cast it into type `String`, and discover what the edit was that triggered the event. As you might imagine, this is a highly customizable component.